

TasteLab

Technical Report: TasteLab Dashboard

By Ayumi Chotoe

Breda University of Applied Sciences
14-1-2026

Executive Summary

The TasteLab Dashboard is a specialised research analytics platform designed to enhance student access to data-driven research on taste and experience. The system integrates synchronised video and audio recordings from experiments and applies custom-built Natural Language Processing (NLP) and Computer Vision (CV) models to automatically extract research insights.

The platform bridges raw data collection and actionable insights through automated speech-to-text transcription, emotion analysis, participant tracking, and human-in-the-loop annotation workflows. This report details the technical implementation, including the Flask architecture, the ETL pipeline, API design, and the challenges overcome during development.

1. Introduction

1.1 Problem Statement

The TasteLab facility, located within the Horizon South Building, serves as a specialised research centre for taste and experience research within the hotel and hospitality domain. The facility works with external stakeholders (including Peter Warner and colleagues from the hospitality industry) to conduct tasting experiments utilising synchronised multimodal data collection through cameras, audio recording, and wearable sensors (eye tracking devices, skin conductance armbands).

Current Challenges:

- **Limited student accessibility:** Data collection and analysis processes are complex and time-consuming, restricting research participation to a small group of students.
- **Inefficient data analysis workflows:** Researchers must manually combine data from multiple tools and wearables, requiring technical expertise beyond most students' skillsets.
- **Annotation bottleneck:** Manual annotation of experimental data is labour-intensive, reducing efficiency and validity.
- **Absence of unified dashboard:** No integrated tool exists to streamline data collection, analysis, and visualisation for taste/experience research.
- **Scalability constraints:** Current approaches cannot scale to support more students conducting independent experiments.

Initial Consideration and Refinement: During project planning, the team considered integrating wearable sensors (eye tracking devices, skin conductance armbands) to capture physiological data. However, after evaluation, wearable data was excluded due to accuracy and synchronization challenges. The project scope was refined to focus exclusively on video and audio analysis, which provides more reliable data for research.

1.2 Solution Overview

We opted for the creation of a web-based dashboard to improve and promote the experience of conducting experiments within TasteLab. The system integrates data from video footage and audio recordings from experiments to provide students, stakeholders and researchers with an intuitive tool for conducting independent research.

The platform incorporates two custom-built analytical models:

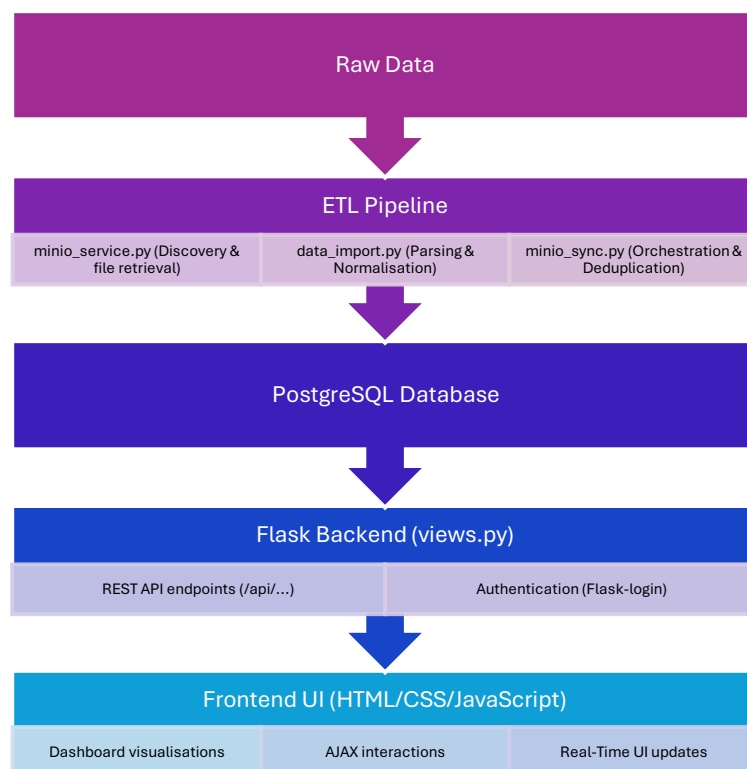
- **Natural Language Processing model:** Processes audio recordings to extract sentiment analysis, emotion detection, speech-to-text transcription, keyword extraction, questions asked, and conversational topic identification.
- **Computer Vision model:** Analyses video recordings to enable multi-camera person detection, participant tracking, and behaviour analysis.

Key capabilities include automated data analysis without technical expertise, unified data integration, non-technical-friendly interface, human-in-the-loop annotation, secure data management, and scalable architecture supporting concurrent experiments.

2. System Architecture

2.1 Architecture Overview

The application follows a modular, layered architecture designed to handle heavy media assets while maintaining a responsive user interface:



2.2 Technology Stack

Component	Technology	Rationale
Frontend	Custom HTML, CSS and JavaScript	Full control over UX/UI No framework overhead
Backend	Flask 3.1.2	Python Integration with data science libraries Lightweight and flexible
Database	PostgreSQL with SQLAlchemy ORM	Relational data with optimised queries Strong integrity constraints
Object Storage	MinIO	Raw analysis files; JSON outputs
Containerisation	Docker & Docker Compose	Consistent deployment across development and production
Orchestration	Portainer.io	Container lifecycle management and easy scaling
Authentication	Flask-Login	Session-based, user role support

2.3 Data Flow Architecture

Key Design Principles:

- **Selective Import:** Only research-relevant structured data is imported; raw pixel data and redundant fields are discarded
- **Batch Processing:** Database operations use bulk insert to handle 100s-1000s of records efficiently
- **Deduplication:** Smart logic prevents duplicate imports when re-running the sync pipeline
- **Error Resilience:** Failed imports roll back completely, allowing safe retries without data corruption

2.4 Frontend-Backend Communication

The dashboard communicates with the Flask backend through REST API endpoints. Key endpoints include:

Experiment Management:

GET /api/experiment/<exp_id>/analysis

→ Returns: NLP metadata, emotion stats, word count, analysis timestamps

GET /api/experiment/<exp_id>/timeline

→ Returns: Timeline segments with emotion, sentiment, and confidence scores

GET /api/experiment/<exp_id>/keywords

→ Returns: Top 20 keywords ranked by TF-IDF score

GET /api/experiment/<exp_id>/tracking_data

→ Returns: Person detection data, participant count, video duration

Sync Operations:

POST /admin/sync-minio

- Triggers manual ETL pipeline
- Returns: Import count, skipped count, execution duration

Annotation Workflow:

GET /api/annotation/jobs

- Lists all annotation labelling jobs

POST /api/annotation/assign

- Assigns the detected person group to the participant ID
- Copies images from raw_crops to labeled_data bucket

POST /api/annotation/undo

- Reverts last annotation assignment

POST /api/annotation/skip

- Moves group to skipped folder

Frontend makes asynchronous requests using fetch API, updating the UI without page reload. Error responses include detailed messages for troubleshooting.

3. Technical Implementation: The ETL Pipeline

3.1 Pipeline Strategy

The ETL pipeline follows a "Sync and Store" strategy rather than real-time queries, reducing latency from 8-12 seconds per chart to 50-200ms after import.

1. **Extract (minio_service.py):** Scans MinIO bucket tastelab-processed-videos for analysis artifacts.
2. **Transform (data_import.py):** Parses JSON, normalises data, validates timestamps.
3. **Load (data_import.py):** Bulk inserts into PostgreSQL (11+ normalised tables).

3.2 Detailed Data Flow

Step 1: Discovery (minio_service.py)

The pipeline scans the MinIO bucket structure following the pattern:

date/session/pipeline_outputs/

Files Identified:

- .chart_data.json - NLP analysis results (sentiment, emotions, transcription)
- .detections.json - Computer vision tracking data (person detection, multi-camera tracking)

Rationale: I chose .chart_data.json rather than .summary.json as the summary is not always detected in the video. Therefore, I opted for .chart_data.json as this is always generated in the NLP model.

Deduplication: When both analysis types exist for a single video, they are grouped under one experiment record to prevent duplicate entries.

Step 2: Intelligent Data Selection

Rather than importing entire 500MB files, the pipeline selectively extracts research-relevant data:

Data Type	What Gets Imported	What's Discarded	Why
Metadata	Duration, participant count, video list	Raw file metadata	Reduces storage load
NLP Analysis	Sentiment, emotions, transcriptions, keywords, topics	Confidence scores below the threshold	Filters to actionable insights
Computer Vision	Multi-camera tracking, person detection	Raw pixel data, intermediate frame data	Raw image data isn't needed for analysis

Step 3: Database Import & Normalisation (data_import.py)

The pipeline creates or updates a normalised database schema comprising 11+ related tables:

Table	Purpose
Experiment	Root record (title, date, participant count, duration, status, tags)
NLPAnalysis	Container for speech/text analysis
TrackingAnalysis	Container for person detection data

Child Tables (automatically created on first import):

- **TimelineSegment:** Emotion breakdown per second of video
- **ChartBin:** Aggregated emotion counts in time buckets (e.g., 10-second intervals)
- **EmotionData:** Overall emotion percentages across entire video
- **TranscriptSummary:** Auto-generated summary of conversation
- **Keyword:** Ranked keywords with TF-IDF scores
- **TopicSentiment:** Conversation topics detected with sentiment
- **DetectedQuestion:** Questions extracted from transcript
- **DetectedAction:** Action items identified from discussion
- **TextInsight:** Complex text analysis (bigrams, trigrams, multi-emotion segments)

Data Normalisation Steps:

1. Emotion vectors: Standardised to a consistent range and scale
2. Timestamps → aligned with transcript base (handles pipeline misalignment)
3. Text cleaning: Folder names sanitised (e.g., “muffin_taste_test_1200” -> “Muffin Taste Test”)
4. Duration validation: Computed from video frames if missing
5. Confidence filtering: Removes predictions below research threshold

3.3 Key Pipeline Optimisations

Bulk Insert Operations

```
db.session.bulk_insert_mappings(TimelineSegment, timeline_segments_data)
```

Instead of looping and inserting records individually, data is batched in groups of 100-1000 records. This reduces database roundtrips and improves write performance by 10-100x for large datasets.

Deduplication Logic: Before importing, the pipeline checks:

- If video_name + session already exists in the database
- Which data types (NLP vs CV) are missing
- If summaries are incomplete

Strategies:

- Skip full import if complete data exists
- Partial update if only one analysis type is missing
- Re-running sync is idempotent (safe to run multiple times)

Timestamp Synchronisation: Emotion vectors and transcripts arrive from separate pipelines with slight misalignments. The sync module:

- Aligns emotion timestamps with transcript events
- Validates all data points fall within the video duration
- Interpolates missing values where needed

3.5 Background Scheduling

The Flask app includes APScheduler configured to run the sync pipeline every 24 hours automatically at 2 AM:

```
@scheduler.task(cron, id='sync_minio', hour=2, minute=0, misfire_grace_time=3600)
def scheduled_minio_sync():
    result = sync_new_analyses(max_imports=None)
```

4. Core Features

4.1 Speech-to-Text & Emotion Analysis

Data Flow

1. Raw video processed by upstream NLP pipeline (external to dashboard)
2. **ETL Imports:** 17 emotions, confidence scores, transcription, timestamps, sentiment, keywords, and topics.
3. Dashboard presents a comprehensive analysis through multiple views:

Dashboard Visualisations:

- **Summary statistics:** Total words, vocabulary size, action items detected, complexity score (0-100).
- **AI summary:** Auto-generated overview of conversation key points.
- **Action items:** Extracted tasks and next steps from dialogue.
- **Key highlights:** Important quotes and memorable statements from the transcript.
- **Complex moments:** Segments where multiple emotions are detected simultaneously (e.g., "happy, hopeful, neutral, sad" at 00:20).

- **Questions asked:** List of detected questions with confidence scores (60-80% range).
- **Full transcript:** Timestamped segments (e.g., 00:20 "Thanks for watching!", 00:36 "Do you understand?").

Researcher Interaction

- Select specific experiments from the dropdown.
- View auto-generated AI summary for quick understanding.
- Review extracted action items for follow-up tasks.
- Analyse complex emotional moments (multi-emotion segments).
- Examine detected questions with confidence metrics.
- Access full timestamped transcript for detailed analysis.
- Filter and search transcript segments.

4.2 Feature 2: Computer Vision (Person Detection & Labelling)

Human-in-the-Loop Workflow

The person detection pipeline combines automatic detection with manual researcher input for accuracy:

Phase 1: Manual Labelling

1. Dashboard displays all people detected by the upstream computer vision pipeline.
2. The researcher manually verifies each detected person.
3. Researcher labels each person (e.g., "Participant A", "Participant B", "Facilitator").
4. Labels stored in the database with associated detection data.

Phase 2: Automatic Model Training

1. Once the labelled dataset reaches the threshold size, the system triggers model training.
2. Custom person detection model trains on labelled data.
3. Model learns participant-specific features (clothing, position, distinctive traits).

Phase 3: Deployment & Analytics

1. Trained model applied to full video.
2. Multi-camera tracking enabled, same person tracked across multiple camera angles
3. Dashboard displays comprehensive movement analytics.

Movement Analytics Visualisation:

- **Summary Metrics:** Total participants, total visits (tracking events), average confidence score (0-1.0 scale), total experiment duration.
- **Movement Heatmap:** Spatial activity levels across camera zones (Food Station, Door, CAM1-4) with colour gradient (low to high activity).
- **Zone Occupancy Table:** Percentage of tracked time spent in each camera zone (e.g., CAM4: 52.6%, CAM1: 31.9%).
- **Participant Tracking Cards:** For each participant, displays:
 - Camera coverage (number of cameras tracking person, e.g., "P001: 4 cams").

- Total tracked duration (e.g., "P001: 4:24").
- Cross-camera movement patterns.

Why This Design: Manual labelling ensures accuracy, automatic model training scales efficiently, and real-time analytics enable researchers to validate participant engagement and movement patterns across the experimental space.

5. Implementation Highlights

5.1 Why Build Custom Instead of Using Existing Tools?

The decision to build a custom Flask-based dashboard was informed by both technical requirements and practical project constraints. The dashboard needed to process multimodal sensory data (audio and video recordings) in a production environment at the BUas facility. Beyond these technical demands, the project operated under several critical constraints: the solution had to be free or low-cost, deployable on university infrastructure, and avoid vendor lock-in. These constraints immediately eliminated several seemingly practical options.

Django

Django is a full-featured framework optimised for large, complex applications with extensive existing infrastructure and established data models. However, TasteLab as a startup research centre, presented a fundamentally different context: minimal existing data systems, no established pipeline infrastructure, and evolving requirements as the facility was still developing its workflows. Implementing Django would have introduced significant architectural overhead, building out models, migrations, permissions systems, and admin configurations, for a project that needed to remain lean and adaptable. Additionally, as someone with limited web development experience, Django's complexity would have created substantial learning friction. Flask, which I had prior experience with, offered a more appropriate scale: lightweight enough to avoid premature complexity, but powerful enough to handle the core requirements of data integration and custom workflow design.

Tableau and Power BI

These business intelligence platforms excel at generating polished visualisations from static datasets. However, they present fundamental misalignments with our requirements. First, licensing costs scale prohibitively; neither platform is designed to be cost-effective for research projects requiring frequent iteration and updates. Second, these tools are fundamentally designed for offline analytics, not integration with active machine learning pipelines. The TasteLab dashboard must seamlessly incorporate real-time computer vision analysis, speech-to-text processing, and emotion detection outputs. These platforms lack the architectural flexibility to serve as a dynamic research toolkit. Both cost and technical constraints eliminated this path.

Streamlit

Streamlit significantly lowers the barrier to building interactive dashboards, making it attractive for rapid prototyping. However, this ease-of-use comes with limitations. Streamlit abstracts away control over UI/UX logic and data handling, making it difficult to implement sophisticated

user authentication, granular permission management, and domain-specific workflows. For a production research toolkit, these constraints became limiting. Additionally, Streamlit deployments typically require paid hosting services, conflicting with our zero-cost infrastructure requirement.

Custom Flask Implementation

Flask emerged as the optimal solution given all project constraints. As a lightweight Python framework, Flask provides complete control over application architecture without imposing constraints or abstractions. This flexibility enables seamless integration with MinIO object storage, PostgreSQL databases, and ML analysis components through direct Python code, a capability unavailable with template-based tools. Equally important, I had prior experience working with Flask, reducing development friction and allowing me to focus on building domain-specific features rather than learning a new framework.

Beyond technical flexibility, Flask aligns perfectly with the project's practical constraints. The framework is free, can be deployed on university servers, integrates easily with free PostgreSQL and MinIO infrastructure, and requires no external hosting or licensing costs. This combination of technical control and financial feasibility made Flask the natural choice.

Benefits of Custom Approach

1. **Domain-specific design:** Every button, chart, and workflow tailored to research needs.
2. **Unique capability:** Direct integration with MinIO and ML pipeline (not possible with generic tools).
3. **Full-stack learning:** Developed a complete application from database to UI.
4. **Flexibility:** Easy to add/modify features as research requirements evolve.
5. **Performance control:** Optimisations tailored to specific data patterns and bottlenecks.

5.2 Development Journey & Architecture Evolution

Critical Constraint: At project inception, TasteLab had not yet conducted experiments, no real data existed to guide dashboard design.

Problem: Database schema design decisions had to be made without knowing the actual data structure, file sizes, or user workflows.

Solution: Close collaboration with a team member developing upstream data pipeline led to the following approach: creating synthetic test data by processing YouTube tasting experiment videos.

Test data: We chose the channel “Epicurious” for our data, as it has a similar tasting experiment structure and would use words related to food more often in the video than any other channel. Which would be meaningful once I analysed the data to identify which features to keep.

Benefits:

- Schema validation before real data arrived
- Edge case discovery (timestamp variations, missing fields, format inconsistencies)

- Workflow optimisation and user testing
- Reduced architectural risk
-

Result: When actual TasteLab experimental data arrived in week 15, the database schema validated successfully with minimal rework.

6. Challenges & Mitigations

Throughout development, several technical challenges were encountered and resolved:

6.1 Data Availability & Schema Uncertainty

Challenge: At project inception, the TasteLab facility had not yet conducted experiments, meaning no real data existed to guide dashboard design. This created fundamental uncertainty about how analysis outputs would be structured, what file sizes to expect, and how users would actually interact with the data.

Impact:

- Database schema design decisions had to be made with incomplete information.
- Risk of architectural rework if assumptions proved incorrect when real data arrived.
- Inability to test workflows and visualisations with realistic data.
- Difficulty estimating storage and performance requirements.

Mitigation Strategy:

- **Synthetic data generation:** Collaborated closely with the team member developing the upstream data pipeline to create realistic test data.
- **YouTube tasting videos:** Identified a public YouTube channel ([link](#)) conducting tasting experiments with a similar structure to planned TasteLab research.
- **NLP Model Testing:** Leveraged the first version of the NLP model to process YouTube videos, generating authentic analysis outputs.
- **Iterative Refinement:** Used synthetic data to validate schema design, identify edge cases, and optimise database structure before real data arrived.
- **Cross-Functional Alignment:** Maintained ongoing communication with the pipeline developer to ensure dashboard expectations aligned with actual outputs.

6.2 Data Duplication

Challenge: Re-running the pipeline for updates frequently resulted in duplicate experiment entries and overlapping analysis records, corrupting historical data.

Impact:

- Inflated experiment counts
- Duplicate rows in analytics tables
- Data integrity issues requiring manual cleanup

Mitigation Strategy:

- Implemented deduplication logic in `minio_sync.py`
- Before import, queries the database to check if the experiment already exists

- Skips files where video_name + session already exist
- Performs partial update if only one data type (e.g., NLP) is missing
- Re-running sync is now idempotent (safe to run multiple times)

6.3 Large Dataset Latency

Challenge: Parsing massive JSON files for TrackingAnalysis (frame-by-frame data with 1000s of records) caused the sync process to timeout, failing large imports.

Impact:

- Unable to process videos longer than 1 hour without a timeout.
- Import process is non-responsive to the user.
- Unpredictable failure modes.

Mitigation Strategy:

- Introduced max_imports parameter to process files in manageable chunks.
- Utilised SQLAlchemy's bulk_insert_mappings to optimise database write speeds.
- Batch processing reduces transaction overhead and memory usage.
- Process now completes successfully for longer videos.

6.4 Unstructured Naming Conventions

Challenge: Raw data folders contained inconsistent naming (timestamps, diverse casing, varied separators), making it difficult to group related experiments.

Impact:

- Same experiment listed multiple times with different names.
- Manual cleanup required.
- Impossible to track experimental relationships.

Mitigation Strategy:

- Developed find_or_create_experiment function using regex patterns.
- Groups varied folder names under a single "Experiment" entity in the database.
- Example: 2026-01-18_tasting-session_01, muffin_test_20260118, Taste_Test_Jan18 all recognized as same experiment.
- Researchers can now view coherent experiment history.

7. Deployment & Performance

7.1 Docker Containerization

The entire application is containerised for consistency:

- Application container: Flask + Python dependencies.
- Database: PostgreSQL (school infrastructure).
- Storage: MinIO (school network/VPN).
- Orchestration: Portainer.io.

Benefits: Environment consistency, easy scaling, dependency freezing, and reproducibility.

8. Limitations & Future Work

Current Limitations

- **MinIO Dependency:** Cannot work offline from the school network.
- **Single Role Model:** All users see all data (not enterprise-ready).
- **VPN Requirement:** Limited accessibility.
- **No Data Export:** Cannot download processed data.

Future Enhancements

Short-term:

- User role system (Admin, Researcher, Viewer)
- Archive feature for read-only experiments
- Data editing capabilities (fix participant count, adjust timestamps)

Medium-term:

- Webhook integration for instant data ingestion
- Incremental sync (re-process only changed files)
- Data export to CSV/JSON/PNG/PDF to review data
- Real-time annotation comments and collaboration tools
- Advanced analytics (experiment comparison, demographic analysis)

9. Conclusion

The TasteLab Dashboard successfully democratizes access to multimodal research analytics through a thoughtfully designed Flask backend and intelligent ETL pipeline. The system transforms 500MB+ analysis files into structured, queryable data with intuitive visualisations.

Key Achievements:

1. Domain-specific solution tailored to sensory research workflows.
2. Automated data pipeline reducing manual handling from hours to one click.
3. Scalable architecture handling videos with optimised queries.
4. Production-ready containerised deployment.
5. Human-in-the-loop ML training integrating researcher expertise.